

STEP 1: Start Hadoop

```
start-dfs.sh  
start-yarn.sh
```

Check:

```
jps
```

👉 Must show:

```
NameNode  
DataNode  
SecondaryNameNode  
ResourceManager  
NodeManager
```

✅ STEP 2: Create Project Folder

```
mkdir ~/hadoop_project  
cd ~/hadoop_project
```

✅ STEP 3: Create Java Program

```
nano LogAnalysis.java
```

Paste this **FULL CODE**:

```
import java.io.IOException;  
import org.apache.hadoop.conf.Configuration;  
import org.apache.hadoop.fs.Path;  
import org.apache.hadoop.io.*;  
import org.apache.hadoop.mapreduce.*;  
import org.apache.hadoop.mapreduce.lib.input.*;  
import org.apache.hadoop.mapreduce.lib.output.*;  
  
public class LogAnalysis {  
  
    public static class Map extends Mapper<LongWritable, Text, Text, IntWritable> {  
        private Text user = new Text();  
        private IntWritable duration = new IntWritable();  
  
        public void map(LongWritable key, Text value, Context context)  
            throws IOException, InterruptedException {  
  
            String line = value.toString();  
            String[] parts = line.split(" ");  
  
            if (parts.length == 3) {
```

```

        user.set(parts[0]);
        int time = Integer.parseInt(parts[2]);
        duration.set(time);
        context.write(user, duration);
    }
}
}

public static class Reduce extends Reducer<Text, IntWritable, Text, IntWritable> {

    public void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {

        int totalTime = 0;

        for (IntWritable val : values) {
            totalTime += val.get();
        }

        context.write(key, new IntWritable(totalTime));
    }
}

public static void main(String[] args) throws Exception {

    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "User Login Time Analysis");

    job.setJarByClass(LogAnalysis.class);
    job.setMapperClass(Map.class);
    job.setReducerClass(Reduce.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);

    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

👉 Save: **CTRL + X → Y → ENTER**

✅ **STEP 4: Create Input File**

nano log.txt

Paste:

```
user1 login 30
user2 login 20
user1 login 40
user3 login 10
user2 login 50
```

Save.

✓ STEP 5: Upload to HDFS

```
hdfs dfs -mkdir /input
hdfs dfs -put log.txt /input
```

Check:

```
hdfs dfs -ls /input
```

✓ STEP 6: Compile Program

⚠ Make sure Java 8 is used

```
javac -classpath `hadoop classpath` -d . LogAnalysis.java
```

✓ STEP 7: Create JAR File

```
jar -cvf loganalysis.jar .
```

✓ STEP 8: Run MapReduce Job

```
hdfs dfs -rm -r /output
hadoop jar loganalysis.jar LogAnalysis /input /output
```

✓ STEP 9: View Output

```
hdfs dfs -cat /output/part-r-00000
```

✓ OUTPUT

```
user1 70
user2 70
user3 10
```

FINAL RESULT

👉 Users with maximum login time:

- **user1**
 - **user2**
-

VIVA (VERY IMPORTANT)

Q: What does Mapper do?

👉 Emits (user, time)

Q: What does Reducer do?

👉 Adds time for each user

Q: What is HDFS?

👉 Distributed storage system

Q: What is MapReduce?

👉 Parallel processing framework

🏁 CONCLUSION

The MapReduce program successfully processed the log file and identified users with maximum login duration.

DONE

You now have:

- ✓ Code
 - ✓ Commands
 - ✓ Output
 - ✓ Explanation
-

If you want, I can also give:

- 📄 Journal write-up (ready to submit)
- 📊 Diagram for exam

Just tell 👍